



SOFTWARE DEV

MERN STACK DEVELOPMENT REPORT

*Week 6 Implementing Global Theme Management Using
React Context API*

Muneeb Ur Rehman

<https://github.com/955muneeb/Glaxit-internship/tree/main/week-6>

Submitted By: Muneeb Ur Rehman

Technology: React.js

Topic: React Context API & `useContext` Hook – Global Theme Management

Objective

The objective of this task was to understand how the **React Context API** can be used to manage global application state without passing data through multiple component levels (prop drilling). A global **Theme Context** was implemented to manage **Light Mode** and **Dark Mode** throughout the application. A toggle button was added to the Navbar to switch between themes, and CSS variables were updated instantly across the application.

Introduction

As React applications grow larger, managing shared data becomes more difficult. Many components may require access to the same data, such as:

- User information
- Authentication status
- Language settings
- Theme (Light/Dark Mode)

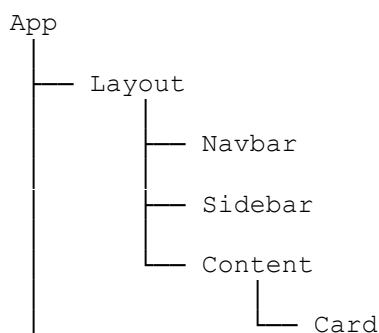
Without Context API, developers pass this data from parent components to child components using **props**. When data must pass through several intermediate components that do not actually use it, the process is called **Prop Drilling**.

React provides the **Context API** as a built-in solution for sharing data globally without manually passing props through every component.

What is Prop Drilling?

Prop drilling is the process of passing data from one component to another through multiple intermediate components.

Example:



Suppose the **Card** component needs the current theme.

Without Context:

```
App
↓
Layout
↓
Content
↓
Card
```

The theme must be passed through every component even if those components never use it.

Example:

```
<App theme={theme}>
↓
<Layout theme={theme}>
↓
<Content theme={theme}>
↓
<Card theme={theme}>
```

This creates unnecessary code and makes maintenance difficult.

Problems with Prop Drilling

- Extra code
 - Poor readability
 - Difficult maintenance
 - Components become tightly coupled
 - Hard to scale large applications
-

React Context API

The Context API allows data to be shared globally without passing props manually.

It is ideal for:

- Themes
 - Authentication
 - Language
 - User Profile
 - Settings
 - Shopping Cart
 - Notifications
-

What is `createContext()` ?

`createContext()` creates a Context object.

Example:

```
import { createContext } from "react";

const ThemeContext = createContext();
```

This object stores shared data that can be accessed by any component inside the Provider.

What is a Provider?

The Provider supplies data to every component inside it.

Example:

```
<ThemeContext.Provider value={value}>
  <App />
</ThemeContext.Provider>
```

Every child component can now access the theme without props.

What is `useContext()` ?

`useContext()` is a React Hook used to read values from a Context.

Example:

```
const { theme } = useContext(ThemeContext);
```

Now the component has direct access to the global theme.

Theme Context Implementation

A Theme Context was created to store:

- Current theme
- Function to change theme

Example:

```
const ThemeContext = createContext();

function ThemeProvider({ children }) {

  const [theme, setTheme] = useState("light");

  const toggleTheme = () => {
```

```
        setTheme(theme === "light" ? "dark" : "light");
    };

    return (
      <ThemeContext.Provider
        value={{ theme, toggleTheme }}
      >
        {children}
      </ThemeContext.Provider>
    );
  }
}
```

Wrapping the Application

The entire application was wrapped inside ThemeProvider.

Example:

```
<ThemeProvider>
  <App />
</ThemeProvider>
```

Now every component can access the theme.

Navbar Theme Toggle

A toggle button was added to the Navbar.

Example:

```
const { theme, toggleTheme } = useContext(ThemeContext);

<button onClick={toggleTheme}>
  Toggle Theme
</button>
```

When clicked:

- Light → Dark
- Dark → Light

The update happens immediately across the application.

CSS Variables

Instead of changing styles individually, CSS Variables were used.

Light Theme:

```
:root {
  --bg-color: white;
  --text-color: black;
  --card-color: #f5f5f5;
```

```
}
```

Dark Theme:

```
.dark {  
  
    --bg-color: #121212;  
    --text-color: white;  
    --card-color: #222;  
  
}
```

Application styles:

```
body{  
  
    background: var(--bg-color);  
    color: var(--text-color);  
  
}  
  
.card{  
  
    background: var(--card-color);  
  
}
```

Whenever the theme changes, the CSS variables automatically update every component.

Workflow of Theme Switching

```
User Clicks Toggle  
    ↓  
toggleTheme() Function  
    ↓  
Theme State Changes  
    ↓  
Context Provider Updates  
    ↓  
All Components Re-render  
    ↓  
CSS Variables Change  
    ↓  
Entire UI Updates
```

Components Using Theme Context

The following components accessed the global theme:

- Navbar
- Home Page
- Cards

- Buttons
- Footer
- Sidebar
- Forms

No props were passed between components.

Advantages of Context API

- Eliminates prop drilling
- Cleaner component hierarchy
- Easier state management
- Better scalability
- Improved maintainability
- Centralized global data
- Built into React (no external library required)

Limitations

Although Context API is powerful, it is not designed for highly complex application state.

For very large projects, developers often use:

- Redux Toolkit
- Zustand
- MobX
- Recoil

However, Context API is an excellent choice for global UI state such as themes and authentication.

Output

Screenshot 1:

Light Theme Interface



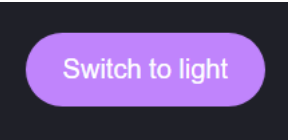
Screenshot 2:

Dark Theme Interface



Screenshot 3:

Navbar Theme Toggle Button



Screenshot 4:

Complete Application After Theme Change

